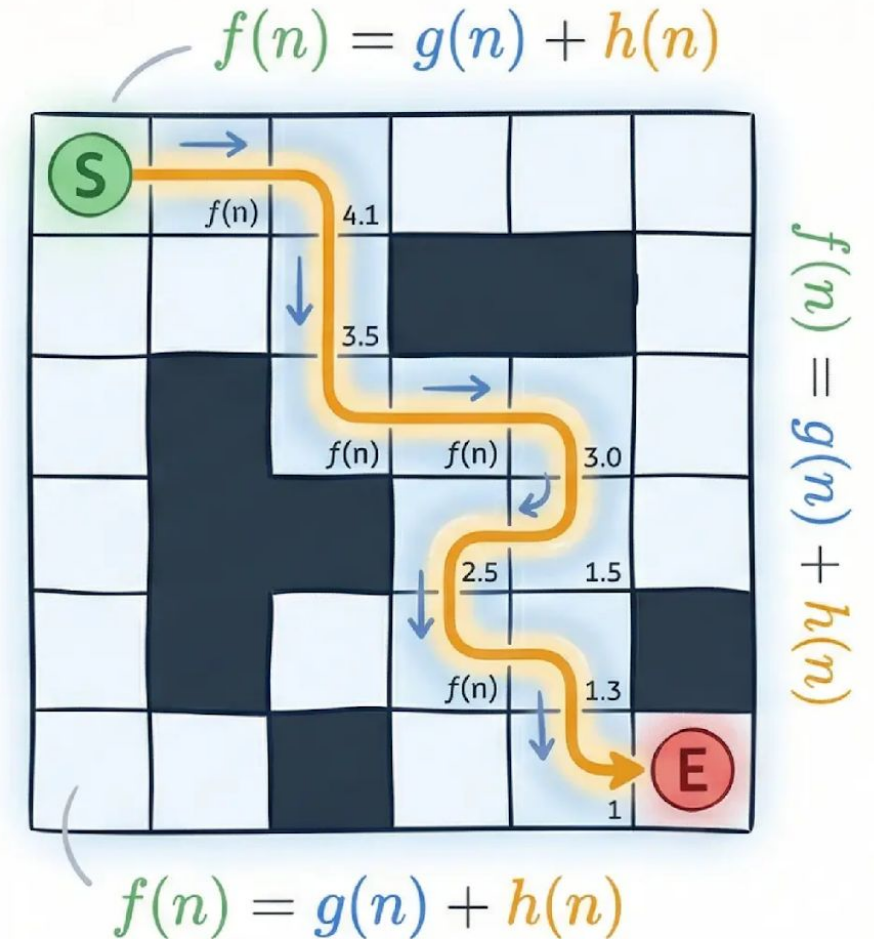


Introduction à l'Algorithme A*

Recherche de chemin optimal,
heuristiques et applications pratiques

📅 6 Avril 2026

👤 Dr Nafaa Haffar



Objectifs Pédagogiques du Cours

À la fin de ce module, vous serez capable de :

01

Comprendre la Formule

Maîtriser le principe de $f(n) = g(n) + h(n)$ et le rôle de chaque composante dans l'évaluation des nœuds.

02

Maîtriser l'Heuristique

Comprendre les propriétés d'admissibilité et de consistance, et savoir choisir une fonction heuristique adaptée.

03

Appliquer à des Problèmes Réels

Résoudre des problèmes concrets de navigation et de recherche de chemin, notamment sur des grilles et labyrinthes.

04

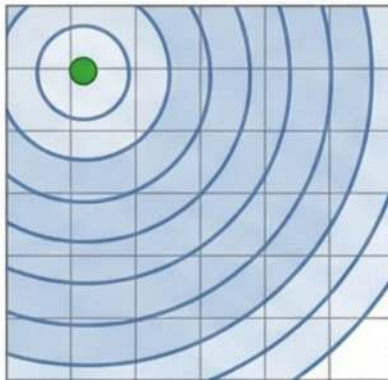
Comparer avec Dijkstra

Situer A* dans le paysage des algorithmes de recherche et comprendre sa supériorité grâce à l'information heuristique.

Dijkstra explore tout. A* explore intelligemment.

Algorithme de Dijkstra

Recherche Non Informée



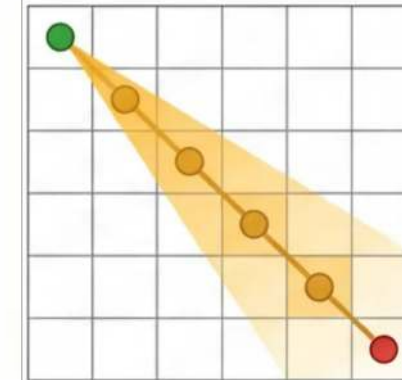
- 📍 Explore les nœuds par coût réel croissant
- 🔄 Aucune connaissance de la cible
- 📊 Garantit l'optimalité, mais explore plus de nœuds

+ Heuristique $h(n)$

➡
Si $h(n) = 0$, A se comporte comme Dijkstra*

Algorithme A*

Recherche Informée



- 🎯 Utilise $f(n) = g(n) + h(n)$ pour guider
- 🧠 L'heuristique $h(n)$ estime la distance restante
- ⚡ Plus rapide grâce à l'exploration ciblée



L'heuristique est la clé : elle permet à A* de "voir" vers où il se dirige, réduisant considérablement le nombre de nœuds explorés.

L'Heuristique : Introduction

*Conditions mathématiques garantissant l'optimalité de A^**

- ❑ Les méthodes **aveugles** (**non informées**) sont des méthodes systématiques peu efficaces
- ❑ Il existe des limites pratiques sur le temps d'exécution et l'espace mémoire pour appliquer ces méthodes sur une grande catégorie de problèmes
- ❑ Toute technique visant à accélérer la recherche est basée sur une information appelée **heuristique**
- ❑ Une heuristique signifie '**aider à découvrir**'
- ❑ Ils utilisent des sources d'information supplémentaires. Ces algorithmes parviennent ainsi à des performances meilleures
- ❑ Les méthodes utilisant des heuristiques sont dites méthodes de recherche heuristiques

L'Heuristique : Introduction

*Conditions mathématiques garantissant l'optimalité de A^**

- Une heuristique doit guider le choix des états à tester et les ordonner selon leurs 'promesses de rapprocher d'un but'.
- ❖ Une heuristique dépend fortement du problème à traiter
- ❖ Une heuristique **pauvre** basée sur des propriétés trop simples du problème sera peu efficace,
- ❖ Une heuristique **riche** basée sur des propriétés approfondies du problème sera efficace, mais est difficile à établir.

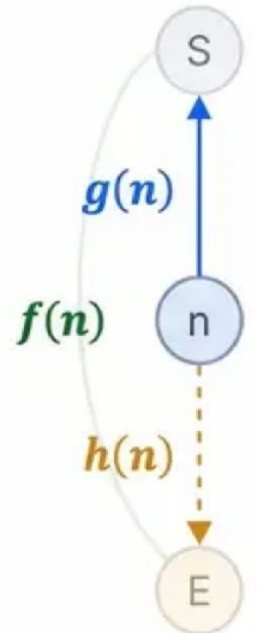
La Formule d'Évaluation

Comment A* évalue chaque nœud pour trouver le chemin optimal

$$f(n) = g(n) + h(n)$$

Coût total estimé = Coût réel parcouru + Estimation du coût restant

Objectif : minimiser $f(n)$. A* sélectionne toujours le nœud ayant la plus petite valeur $f(n)$, ce qui guide efficacement la recherche.



$f(n)$

Coût Total Estimé

Représente l'estimation du coût total du meilleur chemin passant par le nœud n .

A* explore toujours le nœud avec la valeur $f(n)$ la plus faible en premier — c'est ce qui en fait une recherche best-first informée.



$g(n)$

Coût Réel Accumulé

Distance réelle et précise parcourue depuis le nœud de départ jusqu'au nœud actuel n .

Cette valeur est connue et certaine — elle augmente à chaque pas effectué sur le chemin.



$h(n)$

Estimation Heuristique

Estimation du coût restant entre le nœud n et le nœud cible.

Cette valeur est approximative — sa qualité détermine directement la rapidité et l'optimalité de l'algorithme.



L'Heuristique : Admissibilité et Consistance

*Conditions mathématiques garantissant l'optimalité de A^**

$$h(n) \leq \delta(n, \text{cible})$$

**L'heuristique ne doit jamais surestimer le coût réel restant*

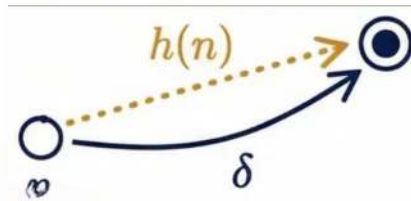
1 Admissibilité

Une heuristique est admissible si, pour tout nœud n :

$$h(n) \leq \delta(n, \text{cible})$$

✓ Garantit que A^* trouve toujours le chemin optimal

"Ne jamais surestimer = toujours rester optimiste sur la distance restante"



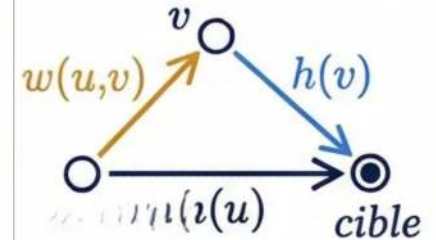
2 Consistance (Monotoniciété)

Une heuristique est consistante si, pour toute arête (u, v) :

$$h(u) \leq h(v) + w(u, v)$$

✓ Implique l'admissibilité — propriété plus forte

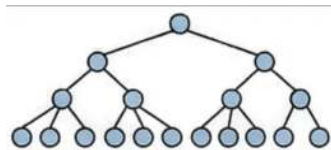
"Consistante $\Rightarrow$ Admissible, mais Admissible $\neq$ Consistante"



Si $h(n) = 0$ pour tout $n \rightarrow A^*$ se comporte comme Dijkstra (optimal mais lent)

Admissibilité et Dominance des Heuristiques

- **Définition de l'admissibilité** : $h(n)$ ne doit jamais surestimer le coût réel vers le but : $h(n) \leq h^*(n)$ pour tout nœud n .
- **Garantie d'optimalité** : Si h est admissible, A^* trouve toujours le chemin optimal en mode "tree-search".
- **Consistance** : condition plus forte : $h(n) \leq \text{coût}(n,m) + h(m)$: garantit l'optimalité en "graph-search" sans ré-explore les nœuds.
- **Concept de dominance** : h_2 domine h_1 si $h_2(n) \geq h_1(n)$ pour tout n , tout en restant admissible.
- **Avantage d'une heuristique dominante** : Une heuristique dominante explore moins de nœuds, rendant A^* plus rapide et plus efficace.
- **Exemple** : tuiles mal placées vs Manhattan : La distance de Manhattan domine le comptage des Tuiles mal placées (h_1)

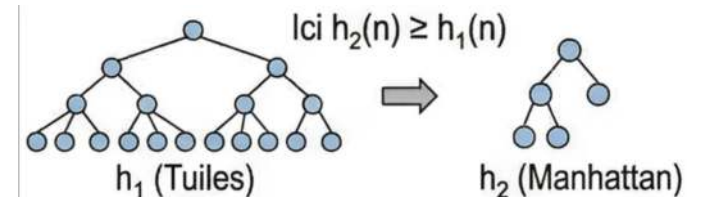


État courant n

0	1	2
3	4	5
6	7	8

Calcul de l'Heuristique

0	1	2
3	4	5
6	7	8



Admissibilité d'une Heuristique

Définition formelle – Une heuristique $h(n)$ est dite **ADMISSIBLE** si et seulement si :

$$h(n) \leq h^*(n) \text{ pour tout nœud } n$$

- où $h^*(n)$ est le **vrai** coût pour aller de n à un état final
- En d'autres termes : $h(n)$ ne surestime JAMAIS le coût réel restant.

Conséquences

Si $h(n)$ surestime $\rightarrow A^*$ peut rater la solution optimale

Si $h(n)$ sous-estime ou est égale $\rightarrow A^*$ garantit l'optimalité

Exemples d'heuristiques admissibles

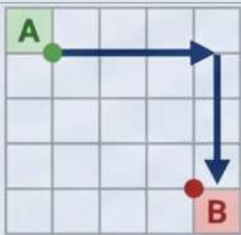
- 8-puzzle : h = nombre de pièces mal placées $\rightarrow$ admissible (chaque pièce nécessite au moins 1 mouvement)
- 8-puzzle : h = distance de Manhattan $\rightarrow$ admissible (chaque pièce doit parcourir au moins cette distance)
- Navigation : h = distance à vol d'oiseau $\rightarrow$ admissible (distance euclidienne $\leq$ distance routière)

Exemple numérique

- Nœud n , $h^*(n) = 5$ (coût réel optimal)
- $h_1(n) = 5 \leq 5$  admissible (parfaite)
- $h_3(n) = 7 > 5$  NON admissible $\rightarrow A^*$ peut échouer à trouver le chemin optimal

Exemples de Fonctions Heuristiques

Choisir $h(n)$ selon les mouvements autorisés dans l'espace de recherche

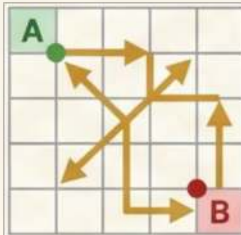


$$h(n) = |x_n - x_B| + |y_n - y_B|$$

Manhattan

Mouvements orthogonaux uniquement
(haut, bas, gauche, droite)
Idéal pour les grilles classiques

✓ Admissible & Consistante

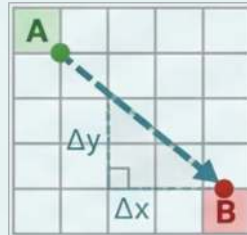


$$h(n) = \max(|x_n - x_B|, |y_n - y_B|)$$

Tchebychev

Mouvements orthogonaux ET diagonaux
autorisés à coût égal
Idéal pour les grilles 8-directionnelles

✓ Admissible & Consistante



$$h(n) = \sqrt{(x_n - x_B)^2 + (y_n - y_B)^2}$$

Euclidienne

Mouvements libres dans toutes directions
Distance "à vol d'oiseau"
Admissible si aucune contrainte de grille

✓ Admissible (mouvement libre)

Mouvements	4 directions	8 directions	Libre
Calcul	Simple	Simple	Modéré
Usage typique	Grilles classiques	Jeux vidéo	Robotique, GPS

Algorithme A* : Procédure et Structures

Pseudo-code de l'algorithme A*

Initialisation

Sommet Initial (I)

Sommet final (B)

Liste des sommets à explorer (OUVERT) : sommet source I

Liste des sommets visités (FERME) : vide

Tant que (la liste OUVERT est non vide) et (B n'est pas dans FERME) Faire

- + Récupérer le sommet X de coût total f minimum.

- + Ajouter X à la liste FERME

- + Ajouter les successeurs de X (non déjà visités) à la liste OUVERT en évaluant leur coût total f et en identifiant leur prédécesseur.

- + Si (un successeur est déjà présent dans OUVERT) et (nouveau coût est inférieur à l'ancien) Alors

 - Changer son coût total

 - Changer son prédécesseur

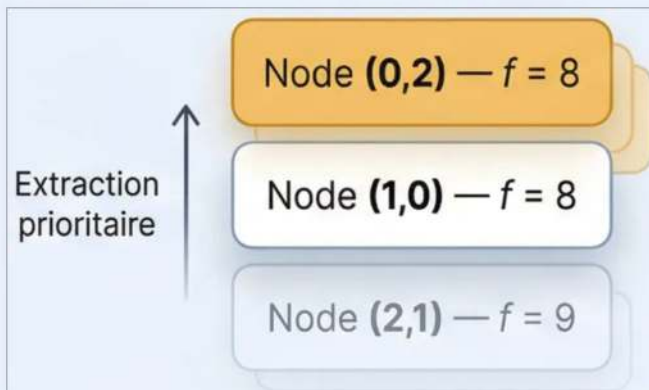
- FinSi

Fin Faire

Structures de Données

Listes Ouverte et Fermée

Liste Ouverte (Open List)



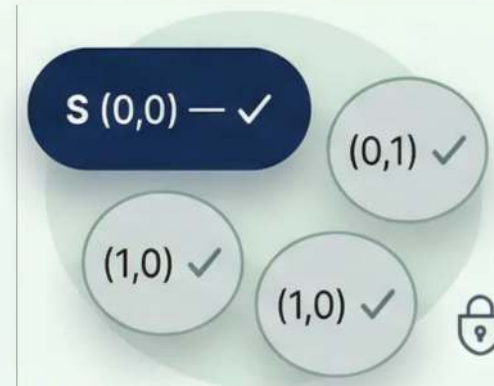
Contient les nœuds découverts mais pas encore explorés.
Implémentée comme une file de priorité — le nœud avec le plus petit $f(n)$ est toujours extrait en premier.

  Triée par $f(n)$ croissant

Exploration terminée →

**Le nœud avec le plus petit $f(n)$ passé de Open à Closed*

Liste Fermée (Closed List)

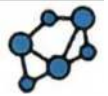


Contient les nœuds déjà visités dont le chemin optimal depuis le départ est définitivement connu.
Évite les calculs redondants et les boucles infinies.

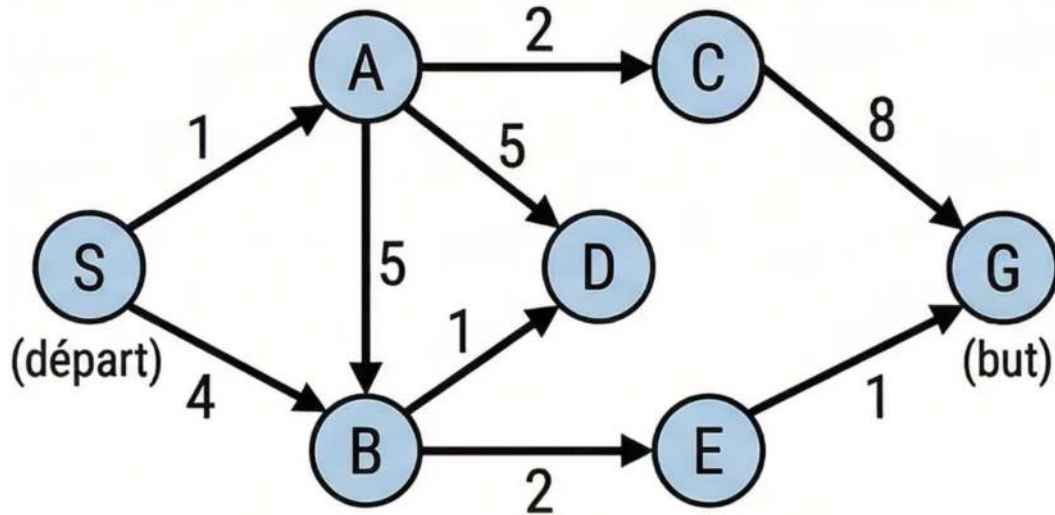
  Nœuds finalisés — ne pas réévaluer

"A explore toujours le nœud le plus prometteur en premier — grâce à la file de priorité."*

Exemple Graphe A* — Étape 0 : Initialisation



Rappel du graphe



Nœud	A	B	C	D	E	h(n)
S(7)	A(6)	B(5)	C(5)	D(3)	E(1)	G(0)

Section 2 — État initial

OPEN = {S}

$$f(S) = g(S) + h(S) = 0 + 7 = 7$$

CLOSED = {}



Explication

- On part du nœud S. Son coût de départ $g(S)=0$ car on y est déjà.
- $h(S)=7$ est l'estimation heuristique vers G.
- Donc $f(S) = g(S) + h(S) = 0 + 7 = 7$.
- S est placé dans OPEN avec $f=7$. CLOSED est vide car aucun nœud n'a encore été exploré.

Exemple Graphe A* — Étapes 1 & 2 : Expansion de S puis A



Étape 1 : Sélectionner S (f=7)

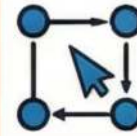
- Sélectionner S (f=7, le plus petit dans OPEN)
- Successeurs de S : A et B

$$f(A) = g(A) + h(A) = (0+1) + 6 = 1 + 6 = 7$$

$$f(B) = g(B) + h(B) = (0+4) + 5 = 4 + 5 = 9$$

• → OPEN = {A:7, B:9} | CLOSED = {S}

💡 On développe S et on calcule f pour chaque voisin direct.



Étape 2 : Sélectionner A (f=7)

- Sélectionner A (f=7, plus petit dans OPEN)
- Successeurs de A : C et D

$$f(C) = g(C) + h(C) = (1+2) + 5 = 3 + 5 = 8$$

$$f(D) = g(D) + h(D) = (1+5) + 3 = 6 + 3 = 9$$

B : g=1+5=6 ✗ (plus mauvais que 4 → ignoré)



• → OPEN = {C:8, B:9, D:9} | CLOSED = {S, A}

A* choisit toujours le nœud avec le plus petit f(n).

$$f(n) = g(n) + h(n)$$

Exemple Graphe A* — Étapes 3 & 4 : Expansion de C puis B

Étape 3 : Sélectionner C (f=8)

Sélectionner C (f=8, plus petit dans OPEN)

Successeurs de C : G

- $f(G) \text{ via } C = g(G) + h(G) = (3+8) + 0 = 11 + 0 = 11$

→ OPEN = {B:9, D:9, G:11} | CLOSED = {S, A, C}

💡 G est trouvé mais avec f=11. A* continue car il peut exister un meilleur chemin (f plus petit dans OPEN).

Étape 4 : Sélectionner B (f=9) — Mise à jour !

Sélectionner B (f=9)

Successeurs de B : D

- D est déjà dans OPEN avec f=9 (via A, g=6)
- Via B : $g(D) = g(B) + \text{coût}(B,D) = 4 + 1 = 5$,
 $f(D) = 5 + 3 = 8 \rightarrow 8 < 9$

MISE À JOUR

ancienne valeur → de D dans OPEN!

- Via E : $g(E) = g(B) + \text{coût}(B,E)$

$$= 4 + 2 = 6, f(E) = 6 + 1 = 7$$

→ OPEN = {E:7, D:8, G:11} | CLOSED = {S, A, C, B}

💡 Mise à jour importante : A* met à jour f(D) car un chemin moins coûteux a été trouvé via B.

Exemple Graphe A* — Étapes 5, 6 & 7 : Vers la Solution

Étape 5 : Sélectionner E (f=7)

Mise à jour G!

Successeurs de E : G

- $f(G)$ via E = $g(G) + h(G) = (6+1) + 0 = 7 + 0 = 7$
- $7 < 11$ (ancienne valeur de G) → MISE À JOUR de G !
- OPEN = {G:7, D:8} | CLOSED = {S, A, C, B, E}
- 💡 Encore une mise à jour de G !

Étape 6: Sélectionner g (f=7), BUT ATTEINT !

Successeurs de G :

👉 On atteint le but → **arrêt** 🎯

- ✓ Chemin optimal trouvé : $S \rightarrow B \rightarrow E \rightarrow G$
- ✓ Coût total = $4 + 2 + 1 = 7$
- ✓ $f(G) = 7 =$ coût optimal ($h(G)=0$ confirmé)

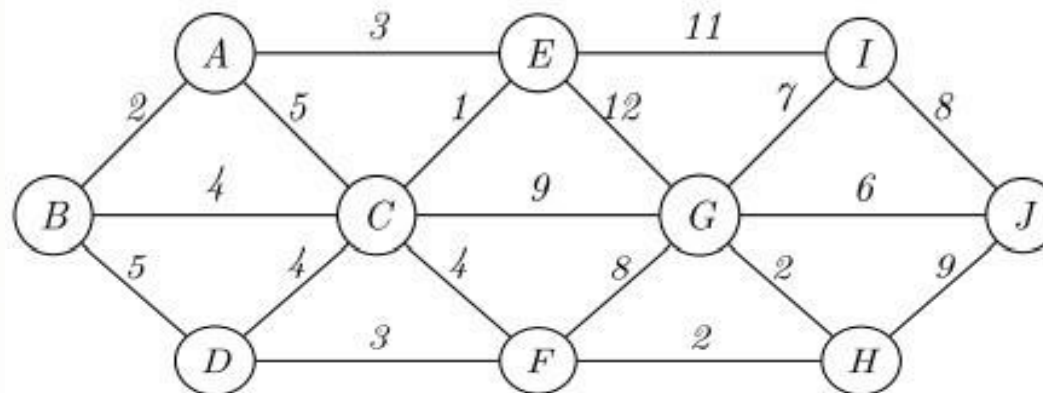
Points clés à retenir



- A* a mis à jour D deux fois (via A puis via B, chemin moins coûteux)
- A* a mis à jour G deux fois (via C puis via E, chemin moins coûteux)
- Le chemin final $S \rightarrow B \rightarrow E \rightarrow G$ est optimal avec coût = 7
- $h(G)=0$ est obligatoire car G est le but (aucun coût restant)

Exercice Pratique : Recherche de Chemin

On considère le graphe suivant où les coûts des arcs représentent les distances entre les sommets. L'objectif est d'utiliser l'algorithme A* pour trouver le chemin optimal de A à H.



Fonction heuristique h pour atteindre H :

A	B	C	D	E	F	G	H	I	J
9	7	3	2	6	1	2	0	4	6

Questions de l'exercice :

1. Appliquez l'algorithme A* sur ce graphe en utilisant l'heuristique h donnée.
 - Détaillez vos calculs en montrant la valeur de $f(n) = g(n) + h(n)$ pour chaque nœud visité.
2. Vérifiez l'admissibilité de l'heuristique h pour atteindre H. Justifiez votre réponse.
3. Donnez le chemin optimal de A à H ainsi que sa valeur trouvée.

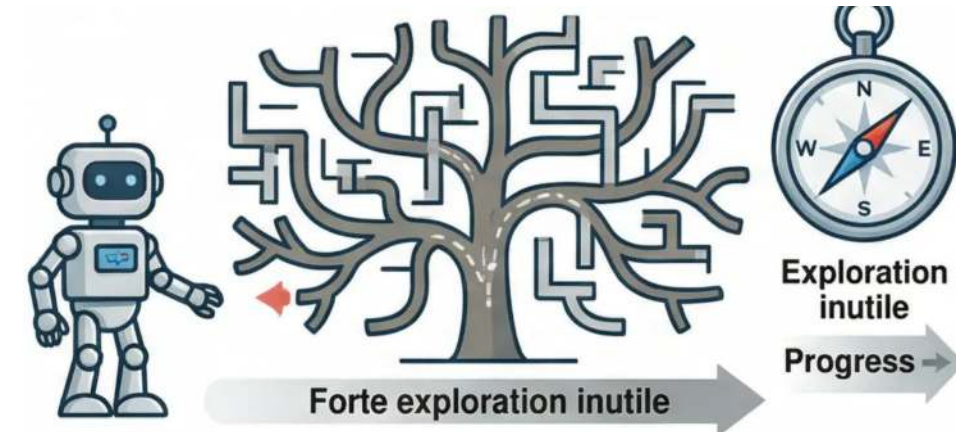
Synthèse et Applications Industrielles

De la théorie à la pratique : A* au cœur des systèmes intelligents

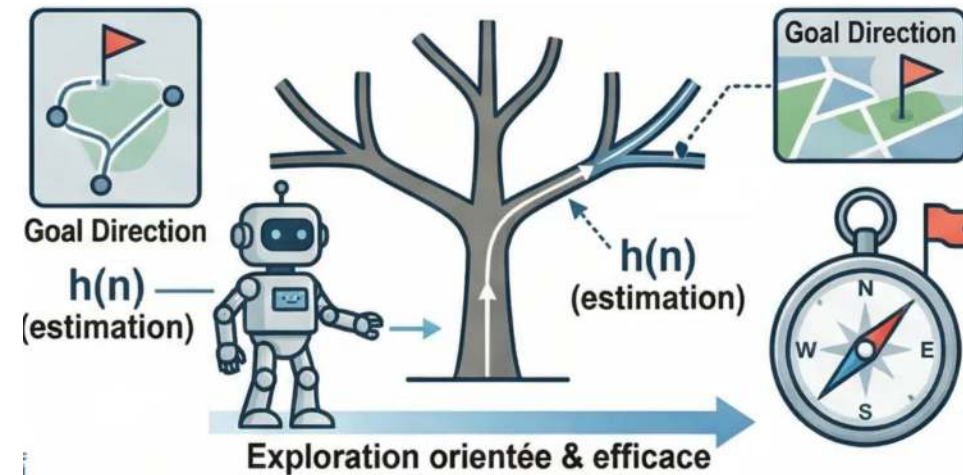
1. **Recherche Aveugle (non informée):**
Explore sans connaissance du but (e.g., BFS, DFS).
Inefficace pour grands espaces d'état.
2. **Recherche Informée (Heuristique):**
Utilise des connaissances spécifiques au problème.
Oriente la recherche vers le but.
3. **Qu'est-ce qu'une Heuristique ?:**
Fonction $h(n)$ estimant le coût restant vers le but.
Une 'supposition éclairée'.
4. **Gain d'Efficacité et de Temps:**
Guide vers les chemins prometteurs.
Moins de nœuds explorés, exécution plus rapide.
5. **Vers l'Algorithme A*:**
Combine coût réel $g(n)$ et heuristique $h(n)$ pour une recherche optimale.



Recherche Aveugle



Recherche Heuristique



Synthèse et Applications Industrielles

De la théorie à la pratique : A* au cœur des systèmes intelligents



Optimalité

Chemin Garanti Optimal

Trouve toujours le chemin le plus court si $h(n)$ est admissible



Efficacité

Recherche Guidée

L'heuristique réduit drastiquement les nœuds explorés



Flexibilité

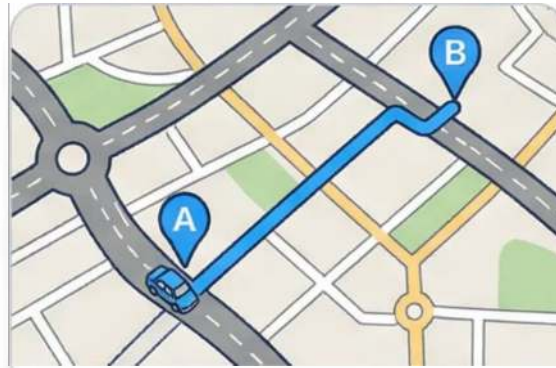
Adaptable à Tout Graphe

Grilles, réseaux routiers, espaces continus...



Jeux Vidéo

Navigation des PNJ dans des environnements complexes



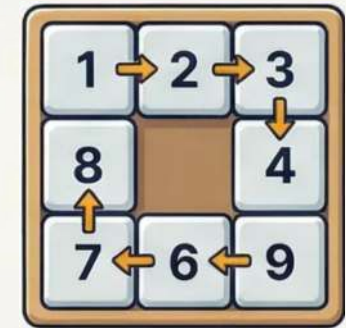
Navigation GPS

Calcul d'itinéraires optimaux en temps réel



Robotique

Planification de trajectoire en environnement dynamique



Puzzles & IA

Taquin, planification d'états, IA symbolique